

OSSP (Operating Systems and Systems Programming) – 25CS2104E

OSSP LAB – PRACTICAL-WEEK 1

Aim

To develop a C program that accepts a Linux command from the user, creates a child process using `fork()`, executes the command using `execvp()`, waits for the child process to complete using `wait()`, and displays the Process IDs (PIDs) of both the parent and child processes

Algorithm

1. Start the program.
2. Read a Linux command from the user.
3. Split the command into the command and its arguments.
4. Call `fork()` to create a child process.
5. If it is the child process:
 - Display the Child Process ID (PID).
 - Execute the command using `execvp()`.
6. If it is the parent process:
 - Display the Parent Process ID (PID).
 - Wait for the child process to finish using `wait()`.
7. Print the completion message.
8. End the program.

Step 1: Open ubuntu and Create a C file

`nano task1.c`

Step 2: Code

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>

int main()
{
    char command[100];
    printf("Enter a Linux command: ");
    scanf("%s", command);
    pid_t pid = fork();
    if(pid < 0)
    {
        printf("Fork Failed!\n");
        return 1;
    }
    else if(pid == 0)
    {
        printf("\nChild Process\n");
        printf("Child PID : %d\n", getpid());
        execlp(command, command, NULL);
        printf("Command execution failed.\n");
    }
    else
    {
        printf("\nParent Process\n");
        printf("Parent PID : %d\n", getpid());
        wait(NULL);
    }
}
```

```
    printf("Child process completed.\n");  
}  
return 0;  
}
```

Step 4: Save the file

Ctrl+o

Enter

Ctrl+x

Step 5: Compile the program

```
gcc task1.c -o task1
```

Step 6: Run the program

```
./task1
```

Using Linux terminal commands (uname , lscpu , lsblk , ps , top)

- Enter a Linux command : `uname`

Displays the Linux kernel name, version, machine architecture, and operating system details.

```

Parent Process
Parent PID : 2000

Child Process
Child PID : 2011
Linux
Child process completed.

```

- Enter a Linux command : lscpu

Displays CPU architecture, number of processors, cores, threads, cache size, and virtualization support.

```

Parent Process
Parent PID : 2028

Child Process
Child PID : 2031

Architecture:
CPU op-mode(s): 32-bit, 64-bit
Address sizes: 39 bits physical, 48 bits virtual
Byte Order: Little Endian
CPU(s): 12
On-line CPU(s) list: 0-11
Vendor ID: GenuineIntel
Model Name: 11th Gen Intel(R) Core(TM) i5-1334U
CPU Family: 6
Model: 186
Thread(s) per core: 2
Core(s) per socket: 6
Socket(s): 1
Stepping: 3
BogoMIPS: 4992.00
Flags: fpu vme de pse tsc mtr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush max fsr ss
e sse2 ss ht syscall nx pdpe1gb rdtscp lm constant_tsc rep_good nopl xtopology tsc_reliable
noinstp.tsc_could_tsc_nocmn.freq pmu pcid mmlqldq vme sse3 fma cx16 pcid sse4.1 sse4.2 x2ap
c movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand hypervisor lahf_lm abm 3dnowpre
fetch_smb sbdr ibre shpb stlb ilbrs enhanced_tsc_shadow opt vpid fsgsbase tsc_adjust bnd
1 avx2 smep bmi2 erms invpcid rdtseed adx smp clflushopt clwb hba_ni xsaveopt xsavec sgxbsv
1 xsave avx_vnni vnni umip waitpkg gfni vaes vpcmlqdq rdpid movdiri movdir64b fsrm md_cl
sae serialize sbt flush_lld arch_capabilities

Virtualization features:
Virtualization: VT-x
Hypervisor vendor: Microsoft
Virtualization type: Full
Caches (sum of all):
L1d: 288 KiB (6 instances)
L1i: 192 KiB (6 instances)
L2: 7.5 MiB (6 instances)
L3: 12 MiB (1 instance)
NUMA:
NUMA node(s): 1
NUMA node CPU(s): 0-11

```

```

NUMA node(s): 1
NUMA node0 CPU(s): 0-11
Vulnerabilities:
Gather data sampling: Not affected
Ghostwrite: Not affected
Indirect target selection: Not affected
Itlb multihit: Not affected
L1tf: Not affected
Mds: Not affected
Meltdown: Not affected
Mmio stale data: Not affected
Old microcode: Not affected
Reg file data sampling: Mitigation; Clear Register File
Retbleed: Mitigation; Enhanced IBRS
Spec rstack overflow: Not affected
Spec store bypass: Mitigation; Speculative Store Bypass disabled via prctl
Spectre v1: Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Spectre v2: Mitigation; Enhanced / Automatic IBRS; IBPB conditional; PBRSE-eIBRS SW sequence; BHI BHI_D
IS_S
Srbds: Not affected
Tsa: Not affected
Tsx async abort: Not affected
Vmscape: Not affected
Child process completed.

```

- Enter a Linux command : lsblk

Lists all storage devices such as hard disks, SSDs, USB drives, and their partitions.

```

Parent Process
Parent PID : 2041

Child Process
Child PID : 2042
NAME MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda   8:0    0 356.9M 1 disk
sdb   8:16    0 159.4M 1 disk
sdc   8:32    0    2G  0 disk [SWAP]
sdd   8:48    0    1T  0 disk /mnt/wslg/distro
/
Child process completed.

```

- Enter a Linux command : ps

Displays all currently running processes along with their Process IDs (PID).

```

Parent Process
Parent PID : 2058

Child Process
Child PID : 2061
  PID TTY          TIME CMD
   372 pts/0        00:00:00 bash
  2058 pts/0        00:00:00 task1
  2061 pts/0        00:00:00 ps
Child process completed.

```

- Enter a Linux command : top

Displays real-time information about CPU usage, memory usage, running processes, and system load.

```
Parent Process
Parent PID : 2348
```

```
Child Process
```

```
Child PID : 2349
```

```
top - 08:37:27 up 1:01, 1 user, load average: 0.08, 0.04, 0.00
```

```
Tasks: 27 total, 1 running, 24 sleeping, 2 stopped, 0 zombie
```

```
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni, 100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
```

```
MiB Mem : 7784.6 total, 6891.1 free, 566.6 used, 483.0 buff/cache
```

```
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used. 7218.1 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1	root	20	0	24116	15092	11288	S	0.0	0.2	0:00.86	systemd
2	root	20	0	3180	2204	2072	S	0.0	0.0	0:00.01	init-systemd(Ub
8	root	20	0	3180	2040	1940	S	0.0	0.0	0:00.00	init
48	root	19	-1	50404	16796	15500	S	0.0	0.2	0:00.31	systemd-journal
80	systemd+	20	0	22416	14516	12020	S	0.0	0.2	0:00.07	systemd-resolve
87	root	20	0	35248	12336	9240	S	0.0	0.2	0:05.50	systemd-udev
109	root	20	0	2888	1952	1820	S	0.0	0.0	0:00.05	chronyd-starter
110	root	20	0	4472	3096	2852	S	0.0	0.0	0:00.02	cron
111	message+	20	0	8788	5492	4780	S	0.0	0.1	0:00.09	dbus-daemon
117	root	20	0	44096	29936	14792	S	0.0	0.4	0:00.23	networkd-dispat
162	root	20	0	18436	9448	8180	S	0.0	0.1	0:00.11	systemd-logind
188	syslog	20	0	220548	5924	4648	S	0.0	0.1	0:00.09	rsyslogd
207	root	20	0	5492	2744	2564	S	0.0	0.0	0:00.00	agetty
235	root	20	0	123460	32616	18004	S	0.0	0.4	0:00.18	unattended-upgr
247	_chrony	20	0	23840	11292	7144	S	0.0	0.1	0:00.17	chronyd
261	_chrony	20	0	12096	2448	1808	S	0.0	0.0	0:00.02	chronyd
368	root	20	0	3184	1108	980	S	0.0	0.0	0:00.00	SessionLeader
370	root	20	0	3200	1248	1108	S	0.0	0.0	0:00.12	Relay(372)
372	bhavana	20	0	6268	5536	3864	S	0.0	0.1	0:00.06	bash
373	root	20	0	8648	5604	4760	S	0.0	0.1	0:00.01	login
428	bhavana	20	0	21940	13112	10904	S	0.0	0.2	0:00.22	systemd
432	bhavana	20	0	22912	4068	2068	S	0.0	0.1	0:00.00	(sd-pam)
459	bhavana	20	0	6136	5408	3856	S	0.0	0.1	0:00.01	bash
2111	bhavana	20	0	2768	1884	1784	T	0.0	0.0	0:00.00	task1
2112	bhavana	20	0	8264	6220	3992	T	0.0	0.1	0:00.79	top
2348	bhavana	20	0	2768	1876	1780	S	0.0	0.0	0:00.00	task1
2349	bhavana	20	0	8172	6092	3988	R	0.0	0.1	0:00.01	top

Conclusion

Linux abstracts hardware resources by providing a simple interface between applications and physical hardware. It efficiently manages CPU scheduling, memory allocation, storage devices, and I/O operations through system services, allowing multiple programs to run safely and efficiently without directly interacting with the hardware.